



UNIVERSITÀ DEGLI STUDI
DI GENOVA

Decentralized System

ASSIGNMENT 1 (NFT – Digital Asset)

Smart Contract Report

Name: Fazeel Ahmed

Matricola: S5640427

Group: Single Member

Submitted To:

Professor Matteo Dell'Amico

Professor Marina Ribaudo

Contents

1. Introduction	3
2. Smart Contract Overview	3
2.1 Contract Imports	3
2.2 Token ID Management	3
2.3 Data Structure	3
DigitalAsset Struct	3
2.4 Mapping for Digital Asset Storage.....	4
2.5 Events	4
3. Smart Contract Functionalities	4
3.1 Creating and Minting an NFT	4
3.3 Listing an NFT for Sale	5
3.4 Removing an NFT from Sale	5
3.5 Purchasing/Buy an NFT	5
4. Compilation and Deployment Process	6
4.1 Compiling the Smart Contract	6
4.2 Deploying on Sepolia Testnet.....	6
4.3 Verifying the Contract on Etherscan	7
6. Conclusion	8

1. Introduction

This report describes the **NFTAssetStore** smart contract, developed in Solidity 0.8.37 using the ERC-721 standard. It allows users to create, list, remove, and purchase NFTs. Each NFT contains details such as title, rarity, traits, price, and metadata URI. The contract is designed for deployment and testing on the **Sepolia Testnet**.

2. Smart Contract Overview

The **NFTAssetStore** uses OpenZeppelin's ERC721URIStorage for NFT creation and metadata storage, and Ownable for contract ownership. It also generates a unique token ID for each NFT.

2.1 Contract Imports

```
import "@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
```

- **ERC721URIStorage**: Provides ERC-721 functionality and allows every NFT to have its own metadata URI.
- **Ownable**: Provides ownership functionality for the smart contract and assigns an owner when the contract is deployed.

2.2 Token ID Management

```
uint256 private nextTokenId = 1;
```

Every newly created NFT receives the current value of `nextTokenId`. After minting, the value is incremented:

```
uint256 tokenId = nextTokenId;
nextTokenId++;
```

This produces unique identifiers such as Token ID 1, Token ID 2, Token ID 3, and so on.

2.3 Data Structure

DigitalAsset Struct

```
struct DigitalAsset {
    uint256 tokenId;
    string title;
    string metadataURI;
    string rarity;
    string traits;
    uint256 initialPrice;
    uint256 sellingPrice;
    address creator;
    bool forSale;
}
```

- **tokenId**: Unique identifier of the NFT.
- **title**: Name or title of the digital asset.

- **metadataURI:** URI pointing to NFT metadata, such as an IPFS URI.
- **rarity:** Rarity of the NFT, for example Common, Rare, or Legendary.
- **traits:** Additional characteristics or attributes of the NFT.
- **initialPrice:** Original price assigned when the NFT is created.
- **sellingPrice:** Current price when the NFT is offered for sale.
- **creator:** Ethereum address of the user who originally created the NFT.
- **forSale:** Indicates whether the NFT is currently available for purchase.

The contract stores `initialPrice` and `sellingPrice` separately. This preserves the original price even if the owner later changes the current sale price.

2.4 Mapping for Digital Asset Storage

```
mapping(uint256 => DigitalAsset) public digitalAssets;
```

The `tokenId` is used as the mapping key. For example, `digitalAssets[1]` returns the stored information associated with NFT token ID 1.

2.5 Events

```
event AssetMinted(uint256 indexed tokenId, address indexed creator, string title, uint256 initialPrice);
event AssetListed(uint256 indexed tokenId, uint256 sellingPrice);
event AssetRemovedFromSale(uint256 indexed tokenId);
event AssetSold(uint256 indexed tokenId, address indexed seller, address indexed buyer, uint256 sellingPrice);
```

- **AssetMinted:** Emitted when a new NFT is successfully created.
- **AssetListed:** Emitted when an NFT owner puts an NFT up for sale.
- **AssetRemovedFromSale:** Emitted when an NFT owner removes an NFT from sale.
- **AssetSold:** Emitted when a buyer successfully purchases an NFT.

These events create blockchain logs and can also be monitored by the frontend application using `ethers.js`.

3. Smart Contract Functionalities

3.1 Creating and Minting an NFT

```
function createAsset( string memory title, string memory metadataURI, string memory rarity,
string memory traits, uint256 initialPrice) public {

    require(initialPrice > 0, "Price must be greater than zero");

    uint256 tokenId = nextTokenId;
    nextTokenId++;

    _safeMint(msg.sender, tokenId);
    _setTokenURI(tokenId, metadataURI);

    digitalAssets[tokenId] = DigitalAsset({attributes values});

    emit AssetMinted(tokenId, msg.sender, title, initialPrice );
}
```

```
}
```

How it works:

- Validates the initial price.
- Generates a unique token ID.
- Mints the NFT to sender (msg.sender).
- Associates the NFT with its metadata URI.
- Stores title, rarity, traits, prices, and creator information.
- Emits an AssetMinted event.

3.3 Listing an NFT for Sale

```
function putForSale(uint256 tokenId, uint256 sellingPrice) public {  
    require(ownerOf(tokenId) == msg.sender, "You are not the owner");  
    require(sellingPrice > 0, "Price must be greater than zero");  
  
    digitalAssets[tokenId].sellingPrice = sellingPrice;  
    digitalAssets[tokenId].forSale = true;  
  
    emit AssetListed(tokenId, sellingPrice);  
}
```

How it works:

- Verifies that the caller owns the NFT.
- Ensures that the selling price is valid.
- Sets the current selling price.
- Marks the NFT as available for sale.
- Emits an AssetListed event.

3.4 Removing an NFT from Sale

```
function removeFromSale(uint256 tokenId) public {  
    require(ownerOf(tokenId) == msg.sender, "You are not the owner");  
    require(digitalAssets[tokenId].forSale, "Asset is not for sale");  
  
    digitalAssets[tokenId].forSale = false;  
    emit AssetRemovedFromSale(tokenId);  
}
```

How it works:

- Checks NFT ownership.
- Confirms that the NFT is currently listed.
- Changes forSale to false.
- Emits an AssetRemovedFromSale event.

3.5 Purchasing/Buy an NFT

```
function purchaseAsset(uint256 tokenId) public payable {
```

```

DigitalAsset storage asset = digitalAssets[tokenId];

require(asset.forSale, "Asset is not for sale");
require(msg.value == asset.sellingPrice, "Incorrect payment amount");

address seller = ownerOf(tokenId);
require(seller != msg.sender, "You already own this asset");

uint256 price = asset.sellingPrice;
asset.forSale = false;

_safeTransfer(seller, msg.sender, tokenId, "");

(bool success, ) = payable(seller).call{value: price}("");
require(success, "Payment failed");

emit AssetSold(tokenId, seller, msg.sender, price);
}

```

How it works:

- Checks that the NFT is listed for sale.
- Requires the exact payment amount.
- Identifies the current seller.
- Prevents a user from purchasing their own NFT.
- Removes the NFT from the selling list.
- Transfers the ERC-721 token to the buyer.
- Transfers the payment to the seller.
- Emits an AssetSold event.

4. Compilation and Deployment Process

4.1 Compiling the Smart Contract

1. Open Remix IDE.
2. Create a file named NFTAssetStore.sol.
3. Add the smart contract code.
4. Open the Solidity Compiler section.
5. Select Solidity compiler 0.8.37.
6. Click Compile NFTAssetStore.sol.
7. Ensure that the contract compiles successfully.

```
pragma solidity ^0.8.37;
```

4.2 Deploying on Sepolia Testnet

8. Connect MetaMask to the Sepolia Test Network.
9. Ensure that the wallet contains sufficient Sepolia test ETH for gas.
10. Open Deploy in Remix.
11. Select the MetaMask-connected environment.

12. Select NFTAssetStore.
13. Click Deploy.
14. Confirm the deployment transaction in MetaMask.
15. Wait for the transaction to be confirmed.
16. Copy the deployed smart contract address.

4.3 Verifying the Contract on Etherscan

17. Go to Sepolia Etherscan and search for the deployed contract address.
18. Click “Verify and Publish.”
19. Select the compiler version 0.8.34.
20. Paste the flattened smart contract code.
21. Submit the contract for verification.

Contract Details on Etherscan:

The screenshot shows the Etherscan Sepolia Testnet interface. At the top, there's a search bar and navigation links for Home, Blockchain, and Tokens. The main section is titled "Transaction Details" and includes a sponsored message from Etherscan API. Below this, there are tabs for Overview, Logs (1), and State. The transaction is identified as a "TRANSACTION ACTION" with the label "Contract Created at" and a specific address. A red warning message states "[This is a Sepolia Testnet transaction only]". The transaction details are listed in a table-like format:

Transaction Hash:	0x41a7815bc91a6d7ae2387a564a83db8afad78c7f467dfa9253df0b06d80ad72c
Status:	Success
Block:	11782482 8219 Block Confirmations
Timestamp:	28 hrs ago Sep-25-2026 11:38:12 PM +UTC
From:	0x4b37EcB2F0E4D0AF2aE8fFC483dDA7B4635Fc180
To:	[0x83216ba405c520ba8d479e283fd81e4a6e0a5e0d Created]
Value:	0 ETH
Transaction Fee:	0.009018673700772476 ETH
Gas Price:	2.553797251 Gwei (0.000000002553797251 ETH)

Transactions Perform on Sepolia Network:

 Home Blockchain Tokens NFTs API More

Contract 0x83216BA405c520bA8D479E283fd81E4a6e0A5E0d

Sponsored: Etherscan API - Onchain data for your agents across 60+ chains via MCP, CLI, Skills. [Build with AI.](#)

Source Code </> API

Overview
ETH BALANCE
0 ETH

More Info
CONTRACT CREATOR
0x4b37EcB2...4635Fc180 | 1 day ago
TOKEN TRACKER
[NFT Asset Store \(NAS\)](#)

Multichain Info
N/A

Transactions Internal Transactions Token Transfers (ERC-20) Contract Events

Latest 3 from a total of 3 transactions Download Page Data

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0x8dbfbfbf52e8...	Purchase Asset	11790300	1 hr ago	0xc0f0b082...cBDbDc8A5	0x83216BA4...a6e0A5E0d	0.0001 ETH	0.00017183
0xb8c5691cbb...	Purchase Asset	11790095	2 hrs ago	0x4b37EcB2...4635Fc180	0x83216BA4...a6e0A5E0d	2 ETH	0.00024458
0x10e598fae85...	Purchase Asset	11789647	3 hrs ago	0x4b37EcB2...4635Fc180	0x83216BA4...a6e0A5E0d	0 ETH	0.00006532

[Download: [CSV Export](#)]

6. Conclusion

The **NFTAssetStore** is an ERC-721 smart contract for creating, listing, and buying NFTs. It stores NFT details and uses IPFS for metadata. The contract can be deployed on the **Sepolia Testnet** and connected to a frontend using **MetaMask** and **ethers.js**.